

Guia Prático de

PromQL

Prometheus Query Language

Este guia foi criado para ajudar qualquer usuário — mesmo sem experiência técnica — a entender e utilizar o PromQL para consultar métricas no Prometheus. Você aprenderá desde o básico até operações avançadas com exemplos práticos.

Conteúdo deste Guia

01	O que é PromQL e como funciona
02	Tipos de métricas e seletores
03	Filtros, funções e operadores
04	Agregações e consultas avançadas

01 — O que é PromQL?

PromQL (Prometheus Query Language) é a linguagem usada para consultar dados de métricas armazenados no **Prometheus**, uma ferramenta de monitoramento muito usada em ambientes de TI. Com PromQL você pode verificar o estado de sistemas, criar alertas e construir dashboards no Grafana.

Como funciona:

O Prometheus coleta métricas de serviços a cada poucos segundos e as armazena com um **timestamp** (data/hora). O PromQL permite buscar e analisar esses dados com consultas simples ou complexas.

■ Exemplo mais simples possível

```
up

# Retorna 1 se o serviço está ativo, 0 se está fora do ar
```

02 — Tipos de Métricas

Existem 4 tipos de métricas no Prometheus. Entender cada tipo é essencial para usar PromQL corretamente:

Tipo	Descrição	Exemplo
Counter	Só sobe. Conta eventos acumulados.	http_requests_total
Gauge	Sobe e desce. Valor atual.	memory_usage_bytes
Histogram	Distribui valores em buckets.	http_duration_seconds
Summary	Percentis pré-calculados.	rpc_duration_seconds

Seletores — Filtrando por Labels

Toda métrica pode ter **labels** (rótulos) que identificam sua origem. Use chaves { } para filtrar:

■ Filtrando com labels

```
http_requests_total{job="api-server"}

# Filtra só pelo job chamado "api-server"

http_requests_total{job="api-server", status="200"}

# Dois filtros ao mesmo tempo (AND implícito)
```

Operador	Significado	Exemplo
=	Igual exato	status="200"
!=	Diferente de	status!="500"
=~	Regex: contém padrão	job=~"api.*"
!~	Regex: não contém	job!~"test.*"

03 — Funções Essenciais

As funções transformam ou calculam valores sobre as métricas. As mais usadas são:

rate() — Taxa de crescimento por segundo

Usada com **Counters**. Calcula quantas vezes por segundo algo aconteceu num intervalo de tempo (range). O intervalo fica entre colchetes `[]`.

■ Usando rate() — requisições por segundo nos últimos 5 min

```
rate(http_requests_total[5m])

# [5m] = janela de 5 minutos
# Resultado: 12.5 req/s → serviço recebendo 12,5 req por segundo
```

■ **Dica:** Use sempre `rate()` com Counters, nunca direto — o valor bruto do counter só cresce e não é útil sozinho.

increase() — Total de aumento no período

Similar ao `rate()`, mas retorna o total de eventos no período, não por segundo.

■ Total de erros na última hora

```
increase(http_errors_total[1h])

# Resultado: 320 → 320 erros ocorreram na última hora
```

irate() — Taxa instantânea (pico)

Usa apenas os 2 últimos pontos. Boa para detectar picos repentinos.

■ Detectar pico de requisições

```
irate(http_requests_total[5m])

# Mais sensível a picos do que rate()
```

histogram_quantile() — Percentis de latência

Responde: *qual é o tempo de resposta que 95% das requisições ficam abaixo?*

■ Latência P95 das requisições HTTP

```
histogram_quantile(0.95, rate(http_duration_seconds_bucket[5m]))

# 0.95 = percentil 95%
# Resultado: 0.32s → 95% das req. respondem em até 320ms
```

Operadores Matemáticos e Comparadores

Operador	Uso	Exemplo
<code>+</code> <code>-</code> <code>*</code> <code>/</code>	Aritmética básica	<code>memory_used / memory_total * 100</code>
<code>==</code> <code>!=</code> <code>></code> <code><</code>	Comparação (filtra)	<code>up == 0 → serviços fora do ar</code>
<code>and</code> <code>or</code> <code>unless</code>	Lógica entre métricas	<code>alertA and alertB</code>

04 — Agregações

Agregações combinam múltiplas séries em uma só. Use **by(label)** para agrupar e **without(label)** para excluir dimensões:

Função	O que faz	Exemplo Prático
sum()	Soma todos os valores	<code>sum(rate(http_req[5m])) by (job)</code>
avg()	Média dos valores	<code>avg(cpu_usage) by (instance)</code>
max()	Valor máximo	<code>max(memory_usage) by (host)</code>
min()	Valor mínimo	<code>min(disk_free) by (mountpoint)</code>
count()	Conta séries	<code>count(up == 1)</code>
topk()	Top N valores	<code>topk(5, rate(errors[5m]))</code>

Exemplos Completos e Práticos

■ CPU em uso (%) por servidor

```
100 - (avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)

# Resultado: instance="srv01" → 73.2% (CPU 73% ocupada)
```

■ Memória disponível em GB

```
node_memory_MemAvailable_bytes / 1024 / 1024 / 1024

# Resultado: 4.32 GB disponíveis
```

■ Porcentagem de erros HTTP (status 5xx)

```
sum(rate(http_requests_total{status=~"5.."}[5m]))
/
sum(rate(http_requests_total[5m])) * 100

# Resultado: 2.1% → 2,1% das requisições estão com erro
```

■ Alertar se serviço ficou fora do ar

```
up{job="pagamentos"} == 0

# Retorna série só se o serviço de pagamentos estiver DOWN
# Use em regras de alerta no AlertManager
```

Referência Rápida — Intervalos de Tempo

s	Segundos	m	Minutos	h	Horas	d	Dias	w	Semanas
[30s]	30 seg	[5m]	5 min	[1h]	1 hora	[7d]	7 dias	[4w]	4 sem

■ **Dica:** Para praticar PromQL sem risco, use o **Prometheus demo público** em demo.prometheus.io ou o **Grafana Play** em play.grafana.org.